Universidad de Costa Rica

CI-0116 Análisis de Algoritmos y Estructuras de Datos / Grupo 2

Tarea Programada 1

Sergio Peralta Barboza

Carlos Manuel Guido Gómez

Julián Farid Vargas Redondo

Sergio Moya

2026

En el presente informe se analizará el comportamiento empírico de complejidad temporal (Big-O) de las seis clases de naves implementadas en la tarea programada. Las 30 batallas simuladas, fueron ejecutadas con naves, y un margen de mejora aleatorio. Dichos datos se encuentran dentro de 'battleLog_INFORME.txt'. Se tiene un registro de 270 naves. Se utilizarán los datos de battleLog.txt para el análisis. Estos son: Total Searches (número de operaciones de búsqueda de una nave por partida), Total Iterations(iteraciones totales ejecutadas por el algoritmo para resolver las búsquedas), Total CPU Time (tiempo total de CPU en microsegundos, para completar las búsquedas). Además, Upgrade History, lista los valores extraídos de una lista para mejorar su rendimiento.

Resumen de Iteraciones, Tiempos y Clase de Nave:

Nave	Big-O	Numero de naves	Promedio Búsquedas	Promedio Iteraciones	Iter/Busq	Promedio CPU(ms)	Min Iteraciones	Max Iteraciones
Nova(Linear)	O(n)	39	132.6	6451.8	48.64	15.24	4247	9108
Vortix(BSL)	O(log n) Worst Case: O(n)	48	124.2	913.4	7.33	8.01	549	1491
Quasar(AVL)	O(log n)	42	141.8	813.0	5.68	9.65	484	1044
Aegis(Set)	O(1)	56	133.0	133.0	1.00	21.45	89	184
Pulsar(Binary Search)	O(log n)	37	130.3	745.9	5.72	15.08	518	1060
Nebula(MinHeap)	O(n)	48	130.7	6300.1	48.23	42.77	4146	9591

Figura 1. Iteraciones promedio por búsqueda

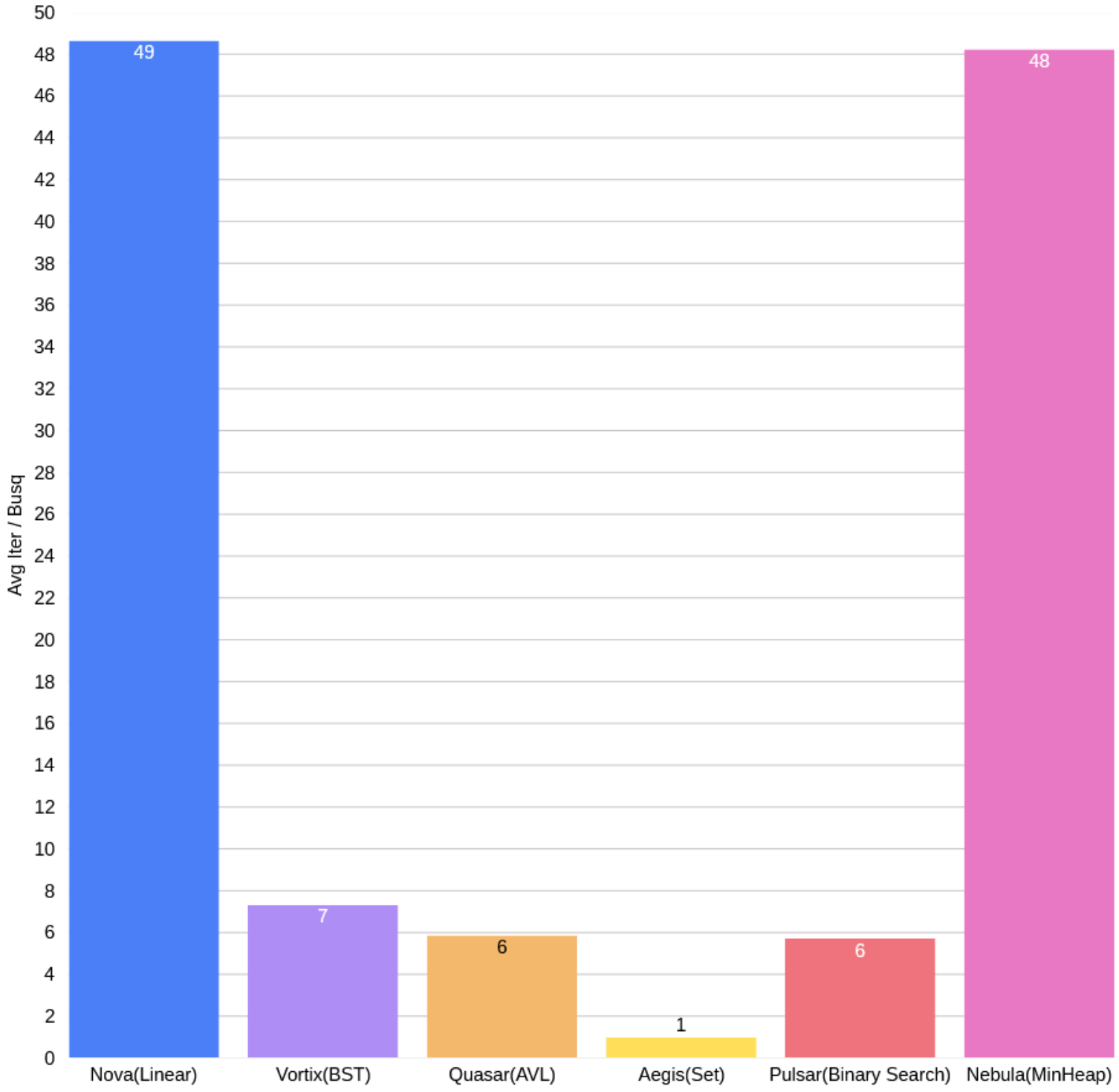
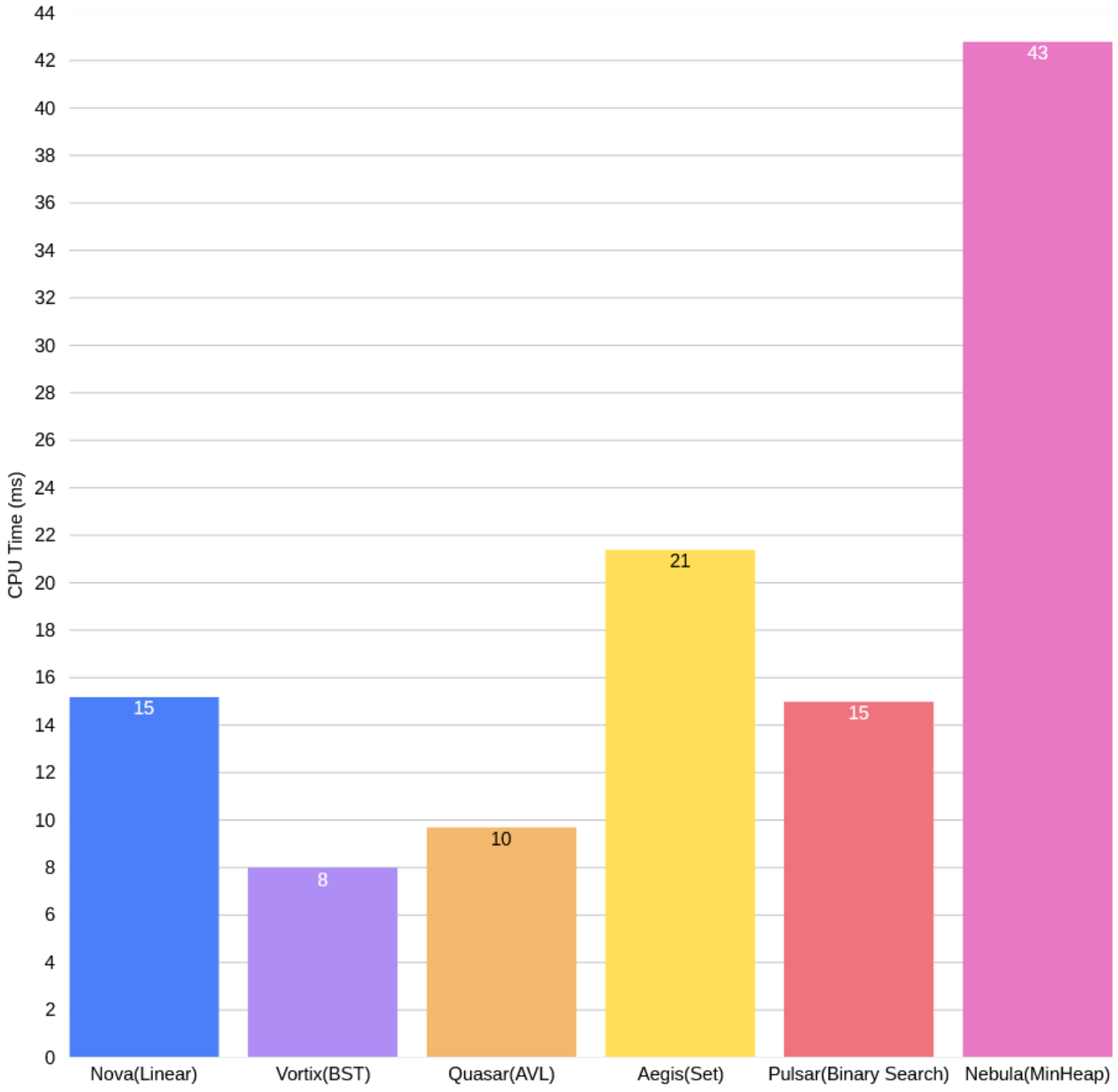


Figura 2. Tiempo promedio de CPU



Analizando la Figura 1 y la Figura 2, se puede notar que las iteraciones promedio por búsqueda no coinciden con su tiempo promedio de CPU. Por ejemplo, Aegis(Set), cuenta con iteraciones promedio de O(1), sin embargo, su complejidad computacional la vuelve el segundo algoritmo que tarda más en el CPU.

Comparación de Rendimiento: AVL, BST y MinHeap:

	Quasar(AVL)	Vortix(BST)	Nebula(MinHeap)
Big-O	O(log n)	O(log n) Worst Case: O(n)	O(n) (busqueda)
Num de naves	42	48	48
Desviacion Estandar de Iteraciones	161.2	210.1	1297.9
Mediana Iteraciones	864	896	5876
Promedio CPU (ms)	9.65	8.01	42.77
Max CPU (ms)	13.4	11.8	63.5
Rango Iteraciones	560	942	5445
Promedio Iter/Busq	5.86	7.33	48.23

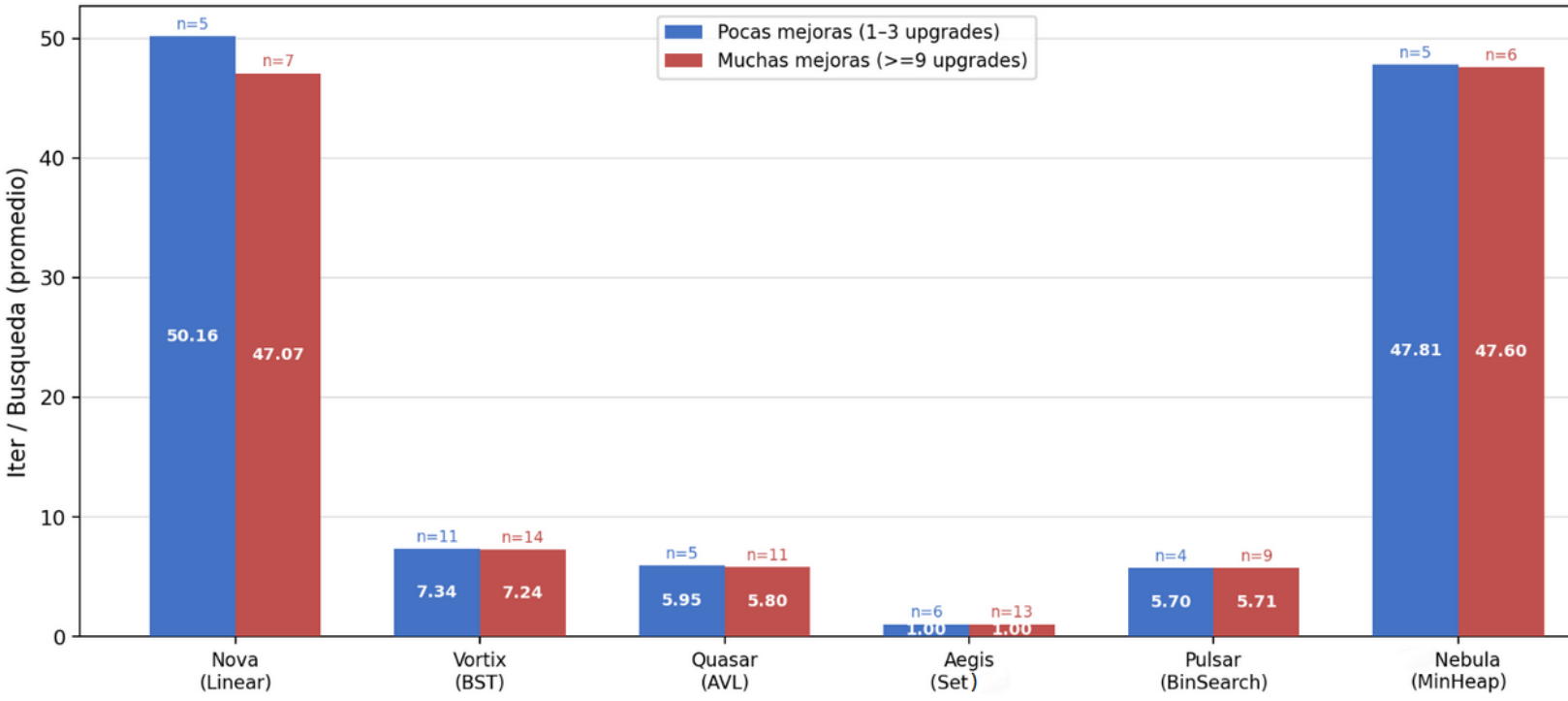
Analizando la tabla de comparación, se puede notar que AVL es más eficiente para búsqueda que BST y MinHeap. Utilizando los datos de Desviación Estándar, mediana y el rango de las iteraciones, se puede notar que MinHeap es muy variable. Los datos de desviación estándar nos indica que tan predecible es el algoritmo, entre más bajo el valor, el algoritmo tiende a ser más estable. Por ende, se puede concluir que el mejor de estos tres algoritmos para búsqueda es el AVL.

Seguidamente, se concluye que MinHeap tiene una altísima variabilidad, es poco predecible, además que su tiempo promedio de CPU es mucho mayor comparado al de los otros algoritmos. También se puede notar que BST es algo más rápido en CPU que AVL. Sin embargo, es más impredecible que AVL, puede degradarse puesto que es más variable.

Impacto Estadístico de las Mejoras en el DPS:

El DPS, se modela directamente con el dato de Iter/Busq. A menor cantidad de iteraciones por búsqueda, el algoritmo encuentra el valor a buscar más rápido. Esto resulta en un mayor daño. El Iter/Busq depende del valor (Big-O) del algoritmo por usar, no del número de upgrades, las mejoras no alteran significativamente la efectividad inicial de los algoritmos, esto se puede evidenciar en la Figura 3.

Figura 3. Impacto de Mejoras en Iter / Busq



(La etiqueta ‘n=’ indica el numero de naves de cada grupo).